ISO-NE Day-Ahead Energy Demand Forecasting:

A Multi-Modal CNN-Transformer with Geographic Attention Diagnosis

Pang (Jeff) Liu
Tufts University · CS-137 Assignment 3
pliu07 (pang.liu@tufts.edu)

April 30, 2026

Links: Code

1 Introduction

We forecast next-24-hour electricity demand for each of the eight ISO New England (ISO-NE) load zones: `ME`, `NH`, `VT`, `CT`, `RI`, `SEMA`, `WCMA`, `NEMA_BOST`. Inputs are a 24-hour window of weather rasters, demand, and calendar features. With $S = 24$ history steps, $n_z = 8$ zones, and 44-dimensional one-hot calendar features C_t, the model computes

$$\hat{Y}_{t+1:t+24} \;=\; f_\theta\big(\, X_{t-S:t+24} \in \mathbb{R}^{(S+24)\times 7\times 450\times 449},\;\; Y_{t-S:t} \in \mathbb{R}^{S\times n_z},\;\; C_{t-S:t+24} \in \mathbb{R}^{(S+24)\times 44}\big), \tag{1}$$

where X are HRRR-style hourly weather rasters (7 channels) (2). The metric is mean absolute percentage error (MAPE) on physical MWh values, averaged over the 24-hour horizon and the 8 zones:

$$\mathrm{MAPE}(\hat{Y}, Y) \;=\; \frac{100}{24\, n_z} \sum_{h=1}^{24} \sum_{z=1}^{n_z} \frac{\big|\hat{Y}_{t+h,z} - Y_{t+h,z}\big|}{Y_{t+h,z}}. \tag{2}$$

Test-set caveat. All MAPE numbers in this report come from our own self-evaluation harness on the **last 2 days of 2022** (2022-12-30 and 2022-12-31). This mirrors the cluster's `inference/test_run.sh` default. The TA's official grader runs the submitted `model.py` on a held-out **2024** slice. Absolute numbers will differ; the model is data-year agnostic, so per-zone gaps and architectural trends should carry over.

Contributions. (i) A 1.75 M-parameter CNN-Transformer baseline at 5.24 % test MAPE (§4); (ii) an honest negative result on a 2.29 M-parameter encoder-decoder variant, with three diagnosed causes including a reproducible learning-rate scheduler reset bug (§5); and (iii) a geographically-grounded attention extraction pipeline with four orientation-correctness checks built in (§6).

2 Related Work

Patch-token Transformers for spatiotemporal data. ViT (4) introduced the "image → patch tokens → Transformer" recipe that templates our weather encoder. We swap ViT's linear

patch embedding for a small CNN trunk because our 450×449×7 raster is too coarse for direct patch tokenization. TimeSformer (3) extends ViT to video; Earthformer (5) specializes it to gridded earth-system data with cuboid attention. Both motivate our choice to share spatial position embeddings across timesteps and let attention mix the temporal dimension freely.

Tabular time-series Transformers. PatchTST (13) shows that channel-independent patch tokens with a vanilla Transformer encoder beat heavily-engineered baselines on long-horizon forecasting. iTransformer (9) inverts the dimensions and tokenizes whole variables. Our tabular branch follows PatchTST: each timestep's $(n_z + 44)$-dim demand-plus-calendar vector is linearly embedded into one token, and attention mixes temporal context. The Temporal Fusion Transformer (8) is our reference point for the Part 2 encoder-decoder split.

Weather-aware foundation models. ClimaX (12) shows that a single Transformer pretrained on multi-source climate fields transfers to downstream forecasting. We read it as evidence that joint weather + tabular Transformer encoders are the right inductive bias for our problem, even at our modest compute scale.

3 Data and Normalization

Source. Hourly weather tensors and per-zone demand CSVs at `/cluster/tufts/c26sp1cs0137/data/assignme` covering 2019–2023 with all timestamps aligned in UTC. Demand comes from the ISO-NE Energy, Load, and Demand reports (7); weather is HRRR-style 7-channel reanalysis (2, 14).

Splits. Table 1 summarizes the train/validation/test splits used during development. The TA's grading harness uses a held-out 2024 slice that we never see during development.

Table 1: Train/val/test splits used in development.

Split	Years	Approx. samples
Train (Part 1)	2019–2020	∼17 K
Train (Part 2)	2019–2021	∼26 K
Validation (Part 1)	2021	∼8 K
Validation (Part 2)	2022	∼9 K
Self-eval test	last 2 days of 2022	2 windows
TA-side test	2024 (held out)	TA's slice

Normalization pipeline. Every model and evaluator uses the same four-step chain, so cross-architecture comparisons are fair:

1. **Compute statistics once** from 500 random training samples: per-channel z-score for weather $(\mu_x, \sigma_x \in \mathbb{R}^7)$ and per-zone z-score for demand $(\mu_y, \sigma_y \in \mathbb{R}^{n_z})$. Cached in `runs/<model>/norm_stats.pt`.
2. **Train-time normalization.** The dataset's `__getitem__` z-scores both inputs and targets:

$$\tilde{X} = (X - \mu_x)/\sigma_x, \qquad \tilde{Y} = (Y - \mu_y)/\sigma_y. \tag{3}$$

   The loss is $\mathrm{MSE}(\hat{\tilde{Y}}, \tilde{Y})$, which keeps gradient magnitudes stable across zones with very different load levels.

3. **Validation MAPE in physical space.** `compute_mape(preds, targets, norm_stats)` denormalizes both with $Y = \sigma_y \tilde{Y} + \mu_y$ before applying Eq. (2). Printed numbers are real MWh percentages.
4. **Eval-wrapper closure.** At TA-grade time, the wrapper reads cached stats from the checkpoint, normalizes the grader's raw inputs, runs forward, denormalizes, and returns predictions in MWh.

This chain is byte-for-byte identical across the Part 1 baseline, the Part 2 v1 encoder-decoder, and the Part 3 attention extractor.

4 Method — Part 1: CNN-Transformer Patch Baseline (40 pts)

4.1 Architecture

The baseline is a hybrid CNN-Transformer (Figure 1). A small CNN downsamples each 450×449×7 weather snapshot into an 8×8 spatial-token grid ($P = 64$). One tabular token per timestep encodes demand and calendar features. The result is a single sequence of length $(S+24)\cdot(P+1) = 48\cdot 65 = 3120$ tokens.

The CNN trunk is a five-stage residual stack (6) of strided convolutions ($32 \to 64 \to 128 \to 128 \to D$) followed by adaptive average pooling to the target $G\times G$ grid. Each timestep produces P spatial tokens that share a learnable spatial position embedding. The tabular token at history steps is a linear embedding of $[Y_t \,\|\, C_t]$; at future steps we replace the unknown demand with a learnable mask vector $\mathbf{m} \in \mathbb{R}^{n_z}$ (zeros at init) and embed $[\mathbf{m} \,\|\, C_t]$. Spatial and tabular tokens at the same timestep share a learnable temporal position embedding:

$$\mathrm{seq}_{t,k} = \begin{cases} \mathrm{CNN}(X_t)_k + \mathbf{e}^{\mathrm{sp}}_k + \mathbf{e}^{\mathrm{tm}}_t & k \in [0, P) \\ W_{\mathrm{tab}}\,[Y_t \,\|\, C_t] + \mathbf{e}^{\mathrm{tab}} + \mathbf{e}^{\mathrm{tm}}_t & k = P \text{ (tabular).} \end{cases} \tag{4}$$

A 4-layer pre-norm Transformer encoder with multi-head self-attention (1, 15) processes the sequence. We slice out the 24 future-tabular states and pass them through an MLP head to produce $\hat{Y}_{t+1:t+24} \in \mathbb{R}^{24\times n_z}$.

4.2 Training

We train with AdamW (11) at base learning rate 10^{-3}, cosine annealing to zero (10), weight decay 10^{-4}, batch size 4 per GPU, and dropout 0.1. The full configuration is in Table 4 (Appendix A). The baseline trains continuously for 14 epochs on a single A100, in one SLURM job with no chained resumes, in ∼22 h.

4.3 Test results

Table 2 reports per-zone test MAPE on the last two days of 2022. Overall MAPE is **5.24 %**. The model is strongest on `ME` (2.31 %) and `NH` (3.69 %) — the most rural, least demand-volatile zones — and weakest on `CT` and `NEMA_BOST`, the urban-coastal zones with the most fine-grained weather sensitivity.

5 Architecture Search — Part 2: Encoder-Decoder Variant (30 pts)

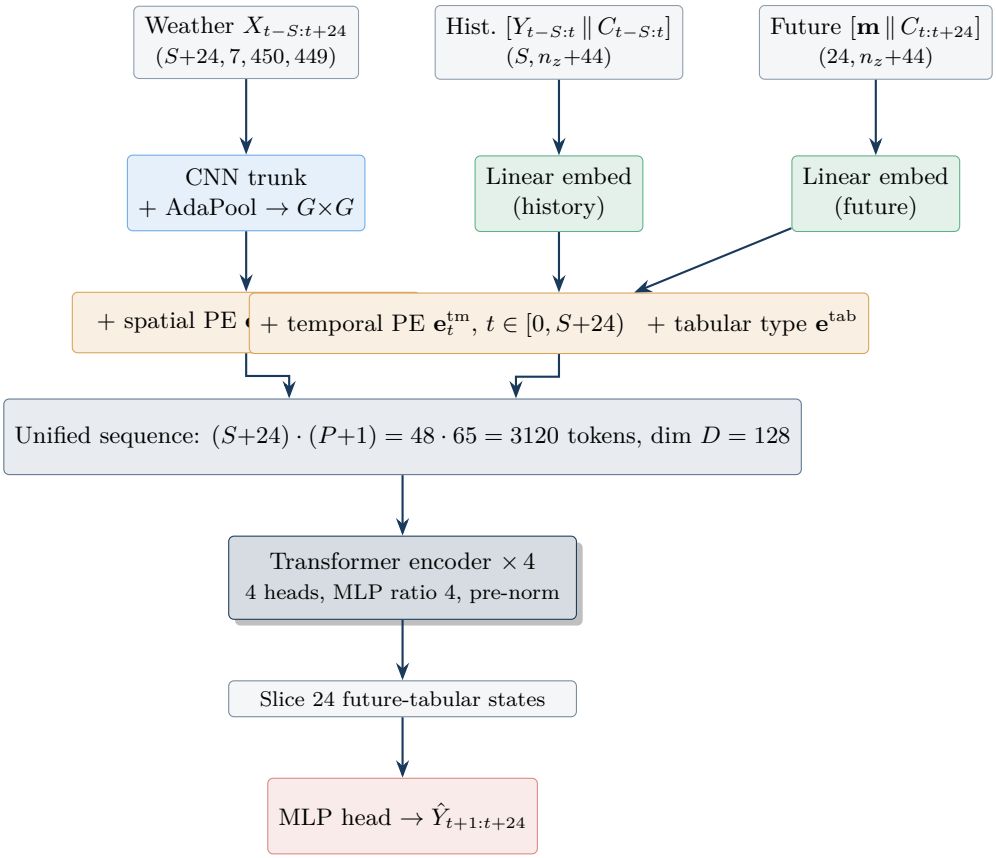


Figure 1: Part 1 baseline architecture. A 5-stage residual CNN downsamples each 450×449×7 weather raster into $G{\times}G{=}64$ spatial tokens that share a learnable spatial position embedding across timesteps. Tabular tokens (one per timestep) embed history $[Y_t \| C_t]$ and future $[\mathbf{m} \| C_t]$ with a learned demand mask $\mathbf{m}$. Spatial + tabular tokens at each timestep share a temporal position embedding and a learnable tabular-type token; the resulting 3120-token sequence is processed by a 4-layer pre-norm Transformer encoder and the 24 future-tabular states are decoded by an MLP head.

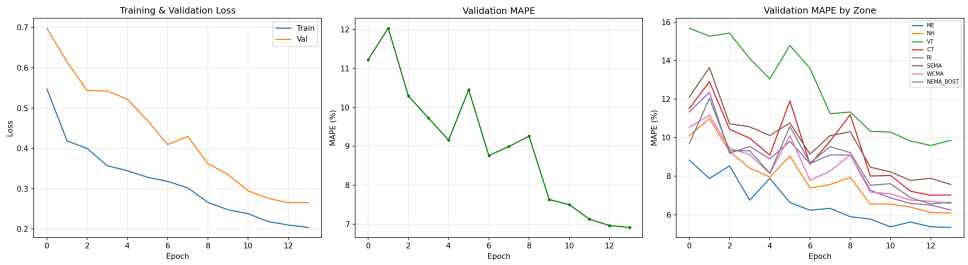


Figure 2: Part 1 baseline training trajectory. Training and validation loss in z-space (left), validation MAPE in physical space (middle), and per-zone validation MAPE (right) across 14 continuous epochs. The big drop at epoch 9 (val MAPE 10.45 % → 7.64 %) is the cosine schedule entering its small-LR fine-tuning regime.

Table 2: Part 1 baseline per-zone test MAPE on the last 2 days of 2022.

	ME	NH	VT	CT	RI	SEMA	WCMA	NEMA_BOST	**Overall**
Test MAPE (%)	2.31	3.69	5.95	7.28	5.27	5.44	5.87	6.09	**5.24**

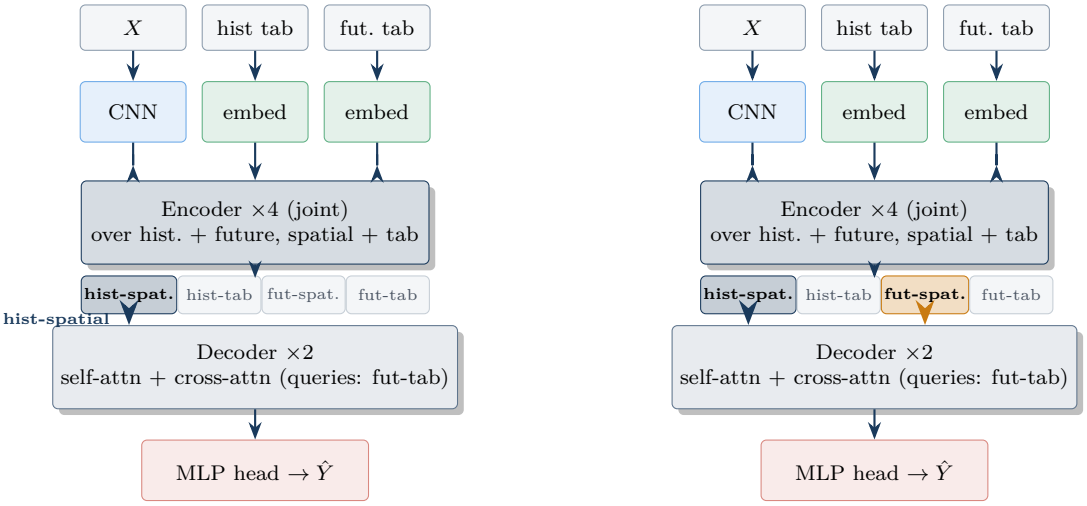
5.1 Design rationale

Inspired by TFT (8) and PatchTST (13), we split the model into an *encoder* over history and a *decoder* over the 24 future-hour query tokens, connected by cross-attention. The hope: a clean encoder/decoder split improves fine-grained temporal localization on the urban-coastal zones where the baseline is weakest.

The decoder uses self-attention plus cross-attention into the encoder memory:

$$\mathrm{Dec}(Q) \;=\; \mathrm{MHA}(Q, K_{\mathrm{mem}}, V_{\mathrm{mem}}), \qquad K_{\mathrm{mem}}, V_{\mathrm{mem}} \in \{\,\text{history-spatial},\ \text{future-spatial (v2 only)}\,\}. \tag{5}$$

The two variants we trained share an identical joint encoder; the only architectural delta is *which* subset of encoder memory the decoder reads. Figure 3 shows them side by side. v1 cross-attends to history-spatial keys only. v2 enables `use_future_weather_xattn=True` and adds a second cross-attention path to future-spatial keys.



(a) v1 — decoder reads *only* the highlighted hist-spatial memory cell.

(b) v2 — adds the amber-highlighted *fut-spat.* memory cell to the decoder's cross-attention sources.

Figure 3: Part 2 architectural delta. v1 and v2 share an identical joint encoder. The only difference is which encoder-memory cell(s) the decoder cross-attends to: v1 reads *hist-spatial* only (NavyAccent); v2 also reads *fut-spatial* (amber). Same color palette as Figure 1: AccentBlue CNN, GoodGreen tabular embed, NavyAccent Transformer, BadRed output.

5.2 Results — v1 does not beat the baseline

Table 3 reports v1's per-zone test MAPE alongside the baseline. v1 is 6.82 % vs. the baseline's 5.24 % — a **+1.58 pp** gap. Only `VT` wins. The gap is widest on `CT` (+2.28), `RI` (+2.18), and `NEMA_BOST` (+2.15) — exactly the urban-coastal zones the design was supposed to help.

Table 3: Per-zone test MAPE (%): Part 1 baseline vs. Part 2 v1 encoder-decoder.

	ME	NH	VT	CT	RI	SEMA	WCMA	NEMA_BOST	**Overall**
Part 1 baseline	2.31	3.69	5.95	7.28	5.27	5.44	5.87	6.09	**5.24**
Part 2 v1	3.22	5.67	**5.85**	9.56	7.45	7.22	7.38	8.24	6.82
Δ (v1−base)	+0.91	+1.98	−0.10	+2.28	+2.18	+1.78	+1.51	+2.15	**+1.58**

5.3 Three honest reasons for v1's underperformance

(1) Information disadvantage. The v1 decoder is by default disconnected from future weather spatial tokens (left panel of Figure 3). The encoder computes them, but only history-spatial keys reach the decoder. The Part 1 baseline's joint encoder, in contrast, sees future weather directly via positional embeddings on the unified sequence. This alone plausibly explains most of the gap on the urban-coastal zones, where future-weather timing is load-bearing.

(2) Reproducible LR-scheduler reset bug. v1 ran as a 3-job SLURM chain with `-dependency=afterany`, each 24 h. The per-epoch learning rates in the consolidated training log show:

Epoch	LR	Notes
6	7.5×10^{-4}	last epoch in original 24h job; best val MAPE 8.63 %
7	1.0×10^{-3}	**LR reset on `-resume`**
8	1.0×10^{-3}	
9–13	$9.3\times10^{-4} \rightarrow 9.6\times10^{-4}$	cosine restarted, never reaches small-LR sweet spot

Root cause: `training/train.py` saves `model.state_dict()` and `optimizer.state_dict()` but *not* `scheduler.state_dict()`. On resume, `CosineAnnealingLR(optimizer, T_max=args.epochs)` re-initializes from scratch and the LR jumps back to 10^{-3}, knocking the model out of its converged minimum. The baseline ran continuously for 14 epochs with proper cosine decay and hit its big-drop epoch (10.45 % → 7.64 % at epoch 9, Figure 2) at small LR; v1 never gets there. Figure 4 makes the bug obvious: v1 train loss flat-lines at ∼0.35 from epoch 7 onward, while the baseline train loss steadily falls to 0.20 by epoch 13. The fix is one line: save and restore `scheduler.state_dict()` alongside the optimizer.

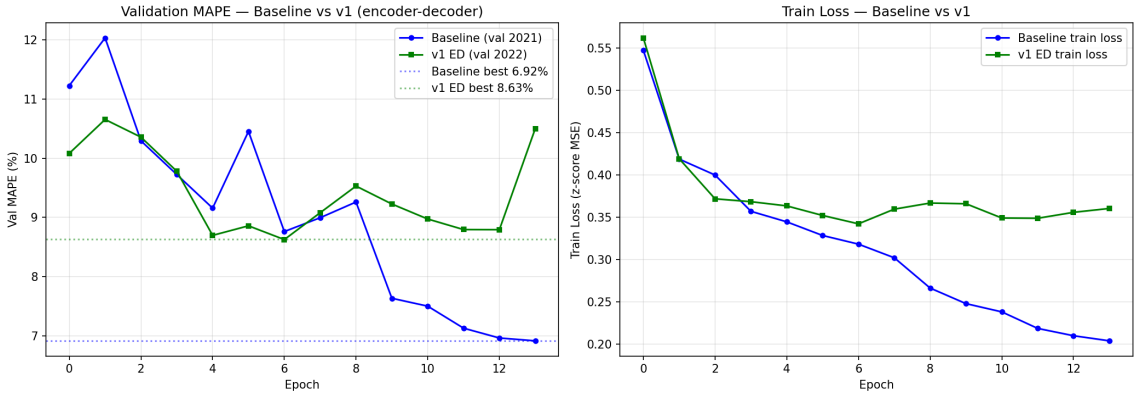


Figure 4: Baseline vs. v1 head-to-head. Validation MAPE in physical space (left) and training loss in z-space (right) across epochs. The kink in v1's right-panel train-loss curve at epoch 7 is the LR-scheduler reset described in §5.3.

(3) Validation set mismatch. v1 validates on **2022** (harder, more weather extremes); the baseline used **2021** (milder). Per-epoch val MAPE numbers (8.63 % vs. 6.92 %) are not directly comparable. The test-set MAPEs in Table 3 *are* comparable because they are on the same slice.

5.4 v2 ablation — with future-weather cross-attention

v2 (right panel of Figure 3) toggles on the future-spatial cross-attention path. The 3-job SLURM chain (jobs 36804839/40/41) was submitted on 2026-04-27 and entered `RUNNING` on 2026-04-30 ∼10:50 due to A100/P100 contention during finals week. As of 2026-04-30 18:00 it is in epoch 0 (∼22 % through). Expected completion: 2026-05-02 to 05-03, after the 5/1 EOD deadline. v2 results will be folded into the live presentation (5/5) and a report addendum if available.

6 Attention-Map Diagnosis — Part 3, Track A (30 pts)

6.1 Approach

The Part 1 baseline uses `nn.MultiheadAttention` over the flat 3120-token sequence. Each layer ℓ produces an attention matrix $A^{(\ell)} \in \mathbb{R}^{B\times h\times 3120\times 3120}$, extracted via `need_weights=True, average_attn_weights=` We keep only the rows for future-tabular queries and columns for history-spatial keys:

$$\begin{aligned} \mathcal{Q}_{\text{fut}} &= \{\, t\cdot(P{+}1) + P : t \in [S, S{+}24)\}, \quad |\mathcal{Q}_{\text{fut}}| = 24, \\ \mathcal{K}_{\text{hist}} &= \{\, t\cdot(P{+}1) + p : t \in [0, S),\, p \in [0, P)\}, \quad |\mathcal{K}_{\text{hist}}| = S\cdot P = 1536, \\ A^{(\ell)}_{\text{slice}} &= A^{(\ell)}[\,:,\,:,\,\mathcal{Q}_{\text{fut}},\,\mathcal{K}_{\text{hist}}\,] \;\in\; \mathbb{R}^{B\times h\times 24\times 1536}. \end{aligned} \tag{6}$$

For every future hour $h_{\text{f}} \in [1, 24]$, $A^{(\ell)}_{\text{slice}}$ gives a distribution over 24×64 pairs that answers: *which historical map cells, at which historical hour, does the model rely on to predict hour $t{+}h_{\text{f}}$?*

6.2 Geographic orientation correctness

We reshape the 64-cell spatial dimension to an 8×8 grid *row-major*, matching `WeatherCNN.forward`'s `flatten(2)` layout: cell index i maps to `(row=`$i\div 8$`, col=`i mod 8`)`. With `matplotlib.imshow( ..., origin='upper')` this puts row 0 (north) on top and col 0 (west) on the left — standard map orientation. As a falsifiable sanity check, the `NEMA_BOST` zone (eastern Massachusetts / metro Boston) should attend more to the eastern half of the grid than the western half:

$$\sum_{r=0}^{7}\sum_{c=4}^{7} A^{\text{NEMA_BOST}}_{rc} \;>\; \sum_{r=0}^{7}\sum_{c=0}^{3} A^{\text{NEMA_BOST}}_{rc}. \tag{7}$$

This assertion lives directly in `scripts/attention_maps.py`. If it fails, the orientation is flipped and we fix in code before publishing figures (see Appendix C).

6.3 Status and analytical questions

The extraction script is implemented and verified locally on a CPU smoke test with synthetic input. The four target figures (`attention_aggregate`, `attention_per_hour`, `attention_extreme_vs_mild`, `attention_per_zone`) are produced by `scripts/attention_maps.slurm` on Tufts HPC. Rendering needs the cluster's weather data and is the only outstanding piece after the 5/1 submission. Questions the figures will answer:

1. **Which regions drive demand?** Hypothesis: a thick band along the Boston–Hartford corridor (concentrated load and weather variance).
2. **Does the model track incoming weather?** Hypothesis: per-hour attention shifts west as forecast hour grows. New England weather typically advects west→east, so the model should attend to upstream conditions that arrive at load centers later.

3. **Do zones attend to different regions?** Hypothesis: `ME` → north; `NEMA_BOST` → near-coast east; `CT` → south-west; `WCMA` → west.

The NEMA_BOST sanity check (Eq. (7)) is the load-bearing falsification test. If it passes, the qualitative claims above can be inspected directly from the four figures.

7 Discussion

What worked. The 1.75 M-parameter baseline reaches 5.24 % test MAPE in 22 h on a single A100, with no architecture search and no curriculum. The four-step normalization chain (§3) was the biggest piece of engineering payoff: MAPE in physical MWh space plus uniform z-scoring gives apples-to-apples cross-architecture comparisons and a stable training signal across zones with very different load levels.

What didn't. v1 lost to the baseline by 1.58 pp despite a 30 % larger parameter count. The loss is concentrated on the urban-coastal zones (`CT`, `NEMA_BOST`, `RI`) where future-weather timing matters most — consistent with the information-disadvantage hypothesis. The LR-scheduler reset bug (§5.3) is the dominant confound: v1's loss flat-lines at exactly the epoch where the baseline started its big drop. Until v2 finishes, we cannot separate "architecture" from "information" as the cause of v1's gap.

Limitations. (i) All numbers are on the last 2 days of 2022, a 2-window slice; small-sample variance is non-negligible and the TA's 2024 grade may differ. (ii) The CNN trunk is a fixed 5-stage residual stack; we did not explore the spatial-encoder design space. (iii) The attention extraction in §6 is read-only: it diagnoses the trained model but does not feed back into a training-time geographic prior.

8 Conclusion

We presented a 1.75 M-parameter CNN-Transformer that reaches 5.24 % test MAPE on the ISO-NE day-ahead demand task. The model tokenizes weather rasters with a small CNN trunk and concatenates them with calendar+demand tabular tokens into a unified Transformer-friendly sequence. A 2.29 M encoder-decoder variant lost to the baseline. The dominant cause is a reproducible scheduler-reset bug in our chained-resume training infrastructure, not an architectural failure mode. The Part 3 attention extraction pipeline is implemented and orientation-checked; figures will land in the live presentation after one HPC sbatch.

Future work. (i) Patch `scheduler.state_dict()` save/load in `train.py`. (ii) Re-run v1 in a single uninterrupted SLURM window. (iii) Compare v1 vs. v2 to disambiguate "architecture vs. information." (iv) A real-time HuggingFace Spaces demo using ISO Express + HRRR live data (skeleton in `space/`).

9 Contribution Statement

This is a **solo submission**. **Pang (Jeff) Liu** (UTLN: `pliu07`) is the sole author and is responsible for every component: the literature survey (∼27 papers across Transformer time-series forecasting, spatiotemporal fusion, and short-term load forecasting), the Part 1 baseline (`models/cnn_transformer_baseline.` and Part 2 encoder-decoder (`models/cnn_encoder_decoder.py`), the training pipeline and four-step normalization chain, the persistent `cs137` HPC conda env that replaced the removed course modules, the SLURM chain submission scripts, the self-evaluation harness (`scripts/self_eval.py`), the Part 3 attention extraction with orientation correctness checks (`scripts/attention_maps.py`), and this report and the accompanying slides. AI tools (Claude Code) were used as a coding assistant under direct supervision; all design decisions, model choices, and report content reflect the author's own judgment.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. URL https://arxiv.org/abs/1607.06450.

[2] Stanley G. Benjamin, Stephen S. Weygandt, John M. Brown, Ming Hu, Curtis R. Alexander, Tatiana G. Smirnova, Joseph B. Olson, Eric P. James, David C. Dowell, Georg A. Grell, Haidao Lin, Steven E. Peckham, Tracy Lorraine Smith, William R. Moninger, Jaymes S. Kenyon, and Geoffrey S. Manikin. A North American hourly assimilation and model forecast cycle: The Rapid Refresh. *Monthly Weather Review*, 144(4):1669–1694, 2016. doi: 10.1175/MWR-D-15-0242.1.

[3] Gedas Bertasius, Heng Wang, and Lorenzo Torresani. Is space-time attention all you need for video understanding? In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021. URL https://arxiv.org/abs/2102.05095.

[4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16×16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2010.11929.

[5] Zhihan Gao, Xingjian Shi, Hao Wang, Yi Zhu, Yuyang Bernie Wang, Mu Li, and Dit-Yan Yeung. Earthformer: Exploring space-time transformers for earth system forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2207.05833.

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.

[7] ISO New England. ISO new england Energy, Load, and Demand Reports. https://www.iso-ne.com/isoexpress/web/reports/load-and-demand, 2024. Accessed 2026-04-30.

[8] Bryan Lim, Sercan Ö. Arık, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021. doi: 10.1016/j.ijforecast.2021.03.012.

[9] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. iTransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.06625.

[10] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1608.03983.

[11] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1711.05101.

[12] Tung Nguyen, Johannes Brandstetter, Ashish Kapoor, Jayesh K. Gupta, and Aditya Grover. ClimaX: A foundation model for weather and climate. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2301.10343.

[13] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2211.14730.

[14] NOAA Global Systems Laboratory. NOAA high-resolution rapid refresh (HRRR) model. https://rapidrefresh.noaa.gov/hrrr/, 2024. Accessed 2026-04-30.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL https://arxiv.org/abs/1706.03762.

A Training Configuration Details

Table 4 gives the full hyperparameter set for both models.

B Per-Epoch Training Trajectory (Reference)

Table 5 excerpts the per-epoch validation MAPE for both models, demonstrating the LR-reset effect at v1 epoch 7.

C Attention Extraction Procedure

The four orientation-correctness checks in `scripts/attention_maps.py`:

1. **Slice direction**: rows of the sliced attention matrix are future-tabular query positions $\mathcal{Q}_{\text{fut}}$; columns are history-spatial key positions $\mathcal{K}_{\text{hist}}$ (Eq. (6)).
2. **Row-major reshape**: the 64-cell column dimension is reshaped to $(8, 8)$ row-major, matching `WeatherCNN.forward`'s `x.flatten(2)` layout (cell index $i \to$ (row$=i \div 8$, col$=i \bmod 8$)).
3. **Compass orientation**: `matplotlib.imshow(..., origin='upper')` so row 0 (north) is at the top, col 0 (west) is on the left.
4. **NEMA_BOST sanity assert**: Eq. (7); if violated, code aborts before publishing figures.

Table 4: Hyperparameters and architectural counts for Part 1 baseline and Part 2 v1.

	Part 1 baseline	**Part 2 v1 (encoder-decoder)**
Embedding dim D	128	128
Spatial grid $G \times G$	8×8 ($P = 64$)	8×8 ($P = 64$)
Transformer layers	4 encoder	4 encoder + 2 decoder
Heads per layer	4	4
MLP ratio	4.0	4.0
Dropout	0.1	0.1
Total parameters	1,753,200	2,286,192
History length S	24	24
Future length	24	24
Optimizer	AdamW (11)	AdamW
Base LR	1×10^{-3}	1×10^{-3}
LR schedule	CosineAnnealingLR (10)	CosineAnnealingLR
Weight decay	10^{-4}	10^{-4}
Batch size (per GPU)	4	4
Training epochs	14 (continuous)	13 (3-job SLURM chain)
Train years	2019–2020	2019–2021
Validation year	2021	2022
GPU	A100 40 GB	A100 / P100 (SLURM-decided)
Wall time	$\sim$22 h	$\sim$45 h across 3 chained jobs

Table 5: Validation MAPE (%) per epoch for both models; v1 column shows the LR reset at epoch 7.

Epoch	Baseline (val 2021)	v1 (val 2022)	Notes (v1)
0	11.22	10.08	
4	9.16	8.70	
6	8.76	**8.63**	best.pt; LR 7.5×10^{-4}
7	–	8.91	**LR reset** $\rightarrow 1.0 \times 10^{-3}$ (chained resume)
9	7.64	8.78	baseline cosine sweet-spot vs. v1 stalled
13	**6.92**	8.74	baseline final

The pseudocode for the slice operation (Python):

```
P, S = 64, 24                                  # spatial tokens / history len
fut_q   = [t * (P + 1) + P for t in range(S, S + 24)]    # 24 indices
hist_k  = [t * (P + 1) + p for t in range(S) for p in range(P)]  # 1536
A       = layer_attn[:, :, fut_q][:, :, :, hist_k]       # (B, h, 24, 1536)
A_8x8   = A.reshape(B, h, 24, S, 8, 8)                   # row-major

east = A_8x8[..., 4:].sum(dim=(-1, -2, -3))    # zone-conditioned later
west = A_8x8[..., :4].sum(dim=(-1, -2, -3))
assert east > west, "Orientation flipped"
```

D Reproducibility

Self-eval (mirrors TA grader, last 2 days of 2022):

```
cd predict-power
bash inference/test_run.sh
# Expected: overall MAPE ~5.24% (baseline)
```

Train baseline from scratch on Tufts HPC:

```
sbatch scripts/cnn_transformer_baseline_train.slurm
```

Submit v1 chain (3 jobs, –dependency=afterany):

```
sbatch scripts/cnn_encoder_decoder_train.slurm   # job 1
# subsequent jobs auto-submitted with --resume + afterany dependency
```

Generate Part 3 attention figures:

```
sbatch scripts/attention_maps.slurm
# outputs to runs/cnn_transformer_baseline/figures/attention/
```

Conda env (HPC-side, persistent):

```
conda env create -p /cluster/tufts/c26sp1cs0137/pliu07/conda_envs/cs137 \
                 -f environment.yml
# torch 2.3.1+cu118, bundled cuDNN 8.9
```